Testing convertibility in parallel

Nathanaëlle Courant

November 14, 2024

Computation

Let $f : (x, y) \mapsto (x - 1)(y + 1)$.
Let's see how we can compute, for instance, $f(1,\ 27 \cdot 31)$.

Computation

Let $f : (x, y) \mapsto (x - 1)(y + 1)$.
Let's see how we can compute, for instance, $f(1,\ 27 \cdot 31)$.

Bad:

$$f(1,\ 27 \cdot 31) \equiv f(1,\ 837)$$

Computation

Let $f : (x, y) \mapsto (x - 1)(y + 1)$.
Let's see how we can compute, for instance, $f(1,\ 27 \cdot 31)$.

Bad:

$$\begin{aligned} f(1,\ 27 \cdot 31) &\equiv f(1,\ 837) \\ &\equiv (1 - 1)(837 + 1) \end{aligned}$$

Computation

Let $f : (x, y) \mapsto (x - 1)(y + 1)$.
Let's see how we can compute, for instance, $f(1,\ 27 \cdot 31)$.

Bad:

$$\begin{aligned} f(1,\ 27 \cdot 31) &\equiv f(1,\ 837) \\ &\equiv (1 - 1)(837 + 1) \\ &\equiv 0 \cdot (837 + 1) \end{aligned}$$

Computation

Let $f : (x, y) \mapsto (x - 1)(y + 1)$.
Let's see how we can compute, for instance, $f(1,\ 27 \cdot 31)$.

Bad:

$$\begin{aligned} f(1,\ 27 \cdot 31) &\equiv f(1,\ 837) \\ &\equiv (1 - 1)(837 + 1) \\ &\equiv 0 \cdot (837 + 1) \\ &\equiv 0 \end{aligned}$$

Computation

Let $f : (x, y) \mapsto (x - 1)(y + 1)$.
Let's see how we can compute, for instance, $f(1,\ 27 \cdot 31)$.

Bad:

$$\begin{aligned} f(1,\ 27 \cdot 31) &\equiv f(1,\ 837) \\ &\equiv (1 - 1)(837 + 1) \\ &\equiv 0 \cdot (837 + 1) \\ &\equiv 0 \end{aligned}$$

Good:

$$f(1,\ 27 \cdot 31) \equiv (1 - 1)(27 \cdot 31 + 1)$$

Computation

Let $f : (x, y) \mapsto (x - 1)(y + 1)$.
Let's see how we can compute, for instance, $f(1,\ 27 \cdot 31)$.

Bad:

$$\begin{aligned} f(1,\ 27 \cdot 31) &\equiv f(1,\ 837) \\ &\equiv (1 - 1)(837 + 1) \\ &\equiv 0 \cdot (837 + 1) \\ &\equiv 0 \end{aligned}$$

Good:

$$\begin{aligned} f(1,\ 27 \cdot 31) &\equiv (1 - 1)(27 \cdot 31 + 1) \\ &\equiv 0 \cdot (27 \cdot 31 + 1) \end{aligned}$$

Computation

Let $f : (x, y) \mapsto (x - 1)(y + 1)$.
Let's see how we can compute, for instance, $f(1,\ 27 \cdot 31)$.

Bad:

$$\begin{aligned} f(1,\ 27 \cdot 31) &\equiv f(1,\ 837) \\ &\equiv (1 - 1)(837 + 1) \\ &\equiv 0 \cdot (837 + 1) \\ &\equiv 0 \end{aligned}$$

Good:

$$\begin{aligned} f(1,\ 27 \cdot 31) &\equiv (1 - 1)(27 \cdot 31 + 1) \\ &\equiv 0 \cdot (27 \cdot 31 + 1) \\ &\equiv 0 \end{aligned}$$

Computation, a different case

Let $f : x \mapsto x \cdot (100 - x)$.
Let's compute $f(7 \cdot 13 - 1)$.

This time it was better to compute the argument first!

Computation, a different case

Let $f : x \mapsto x \cdot (100 - x)$.
Let's compute $f(7 \cdot 13 - 1)$.

Bad:

$$f(7 \cdot 13 - 1)$$
$$\equiv (7 \cdot 13 - 1)(100 - (7 \cdot 13 - 1))$$

This time it was better to compute the argument first!

Computation, a different case

Let $f : x \mapsto x \cdot (100 - x)$.
Let's compute $f(7 \cdot 13 - 1)$.

Bad:

$$
\begin{aligned}
& f(7 \cdot 13 - 1) \\
& \equiv (7 \cdot 13 - 1)(100 - (7 \cdot 13 - 1)) \\
& \equiv (91 - 1)(100 - (7 \cdot 13 - 1)) \\
& \equiv 90 \cdot (100 - (7 \cdot 13 - 1))
\end{aligned}
$$

This time it was better to compute the argument first!

Computation, a different case

Let $f : x \mapsto x \cdot (100 - x)$.
Let's compute $f(7 \cdot 13 - 1)$.

Bad:

$$
\begin{aligned}
& f(7 \cdot 13 - 1) \\
& \equiv (7 \cdot 13 - 1)(100 - (7 \cdot 13 - 1)) \\
& \equiv (91 - 1)(100 - (7 \cdot 13 - 1)) \\
& \equiv 90 \cdot (100 - (7 \cdot 13 - 1)) \\
& \equiv 90 \cdot (100 - (91 - 1)) \\
& \equiv 90 \cdot (100 - 90)
\end{aligned}
$$

This time it was better to compute the argument first!

Computation, a different case

Let $f : x \mapsto x \cdot (100 - x)$.
Let's compute $f(7 \cdot 13 - 1)$.

Bad:

$$
\begin{aligned}
& f(7 \cdot 13 - 1) \\
& \equiv (7 \cdot 13 - 1)(100 - (7 \cdot 13 - 1)) \\
& \equiv (91 - 1)(100 - (7 \cdot 13 - 1)) \\
& \equiv 90 \cdot (100 - (7 \cdot 13 - 1)) \\
& \equiv 90 \cdot (100 - (91 - 1)) \\
& \equiv 90 \cdot (100 - 90) \\
& \equiv 90 \cdot 10 \\
& \equiv 900
\end{aligned}
$$

This time it was better to compute the argument first!

Computation, a different case

Let $f : x \mapsto x \cdot (100 - x)$.
Let's compute $f(7 \cdot 13 - 1)$.

Bad:

$$
\begin{aligned}
& f(7 \cdot 13 - 1) \\
& \equiv (7 \cdot 13 - 1)(100 - (7 \cdot 13 - 1)) \\
& \equiv (91 - 1)(100 - (7 \cdot 13 - 1)) \\
& \equiv 90 \cdot (100 - (7 \cdot 13 - 1)) \\
& \equiv 90 \cdot (100 - (91 - 1)) \\
& \equiv 90 \cdot (100 - 90) \\
& \equiv 90 \cdot 10 \\
& \equiv 900
\end{aligned}
$$

Good:

$$
\begin{aligned}
& f(7 \cdot 13 - 1) \\
& \equiv f(91 - 1) \\
& \equiv f(90)
\end{aligned}
$$

This time it was better to compute the argument first!

Computation, a different case

Let $f : x \mapsto x \cdot (100 - x)$.
Let's compute $f(7 \cdot 13 - 1)$.

Bad:

$$
\begin{aligned}
& f(7 \cdot 13 - 1) \\
& \equiv (7 \cdot 13 - 1)(100 - (7 \cdot 13 - 1)) \\
& \equiv (91 - 1)(100 - (7 \cdot 13 - 1)) \\
& \equiv 90 \cdot (100 - (7 \cdot 13 - 1)) \\
& \equiv 90 \cdot (100 - (91 - 1)) \\
& \equiv 90 \cdot (100 - 90) \\
& \equiv 90 \cdot 10 \\
& \equiv 900
\end{aligned}
$$

Good:

$$
\begin{aligned}
& f(7 \cdot 13 - 1) \\
& \equiv f(91 - 1) \\
& \equiv f(90) \\
& \equiv 90 \cdot (100 - 90)
\end{aligned}
$$

This time it was better to compute the argument first!

Computation, a different case

Let $f : x \mapsto x \cdot (100 - x)$.
Let's compute $f(7 \cdot 13 - 1)$.

Bad:

$$
\begin{aligned}
& f(7 \cdot 13 - 1) \\
& \equiv (7 \cdot 13 - 1)(100 - (7 \cdot 13 - 1)) \\
& \equiv (91 - 1)(100 - (7 \cdot 13 - 1)) \\
& \equiv 90 \cdot (100 - (7 \cdot 13 - 1)) \\
& \equiv 90 \cdot (100 - (91 - 1)) \\
& \equiv 90 \cdot (100 - 90) \\
& \equiv 90 \cdot 10 \\
& \equiv 900
\end{aligned}
$$

Good:

$$
\begin{aligned}
& f(7 \cdot 13 - 1) \\
& \equiv f(91 - 1) \\
& \equiv f(90) \\
& \equiv 90 \cdot (100 - 90) \\
& \equiv 90 \cdot 10 \\
& \equiv 900
\end{aligned}
$$

This time it was better to compute the argument first!

Equality, up to computation

Let $f : x \mapsto 2^x$.
Let's prove $f(20) \equiv f(19+1)$.

First idea: let's compute both sides.

$$f(20) \equiv 2^{20} \equiv 1048576$$

$$f(19+1) \equiv 2^{19+1} \equiv 2^{20} \equiv 1048576$$

Costly: we computed 2^{20} on both sides, which was expensive.

Equality, up to computation

Let $f : x \mapsto 2^x$.
Let's prove $f(20) \equiv f(19 + 1)$.

Second idea: if $x \equiv y$, then $f(x) \equiv f(y)$, so we only need to prove $20 \equiv 19 + 1$.

$$20 \equiv 20 \qquad\qquad 19 + 1 \equiv 20$$

That's it, cheap computations only!

Equality, up to computation: the bad case

Let $f : (x, y) \mapsto (x - 1)(y + 1)$.
Let's prove $f(1,\ 27 \cdot 31) \equiv f(1,\ 19 \cdot 47)$.

Let's do as before, and prove $1 \equiv 1$ and $27 \cdot 31 \equiv 19 \cdot 47$.

$$27 \cdot 31 \equiv 837 \qquad\qquad 19 \cdot 47 \equiv 893$$

They are different, so this didn't work!

Equality, up to computation: the bad case

Let $f : (x, y) \mapsto (x - 1)(y + 1)$.
Let's prove $f(1,\ 27 \cdot 31) \equiv f(1,\ 19 \cdot 47)$.

Let's do as before, and prove $1 \equiv 1$ and $27 \cdot 31 \equiv 19 \cdot 47$.

$$27 \cdot 31 \equiv 837 \qquad\qquad 19 \cdot 47 \equiv 893$$

They are different, so this didn't work! We need to unfold f and compute both sides. We already computed both arguments so we don't need to compute them again:

$$\begin{aligned} f(1,\ 837) &\equiv (1 - 1)(837 + 1) \\ &\equiv 0 \cdot (837 + 1) \\ &\equiv 0 \end{aligned} \qquad \begin{aligned} f(1,\ 893) &\equiv (1 - 1)(893 + 1) \\ &\equiv 0 \cdot (893 + 1) \\ &\equiv 0 \end{aligned}$$

We still computed $27 \cdot 31$ and $19 \cdot 47$: expensive.

Equality, up to computation: the bad case

Let $f : (x, y) \mapsto (x - 1)(y + 1)$.
Let's prove $f(1,\ 27 \cdot 31) \equiv f(1,\ 19 \cdot 47)$.

Instead, let's unfold f first.

$$\begin{aligned} f(1,\ 27 \cdot 31) &\equiv (1 - 1)(27 \cdot 31 + 1) \\ &\equiv 0 \cdot (27 \cdot 31 + 1) \\ &\equiv 0 \end{aligned} \qquad \begin{aligned} f(1,\ 19 \cdot 47) &\equiv (1 - 1)(19 \cdot 47 + 1) \\ &\equiv 0 \cdot (19 \cdot 47 + 1) \\ &\equiv 0 \end{aligned}$$

A lot better! But how to know which we should do?
Short answer: we don't, so rely on heuristics (previous works)...
or do it in parallel!

The problem

λ-calculus with defined constants c, each constant is associated to a term.

Two rules:

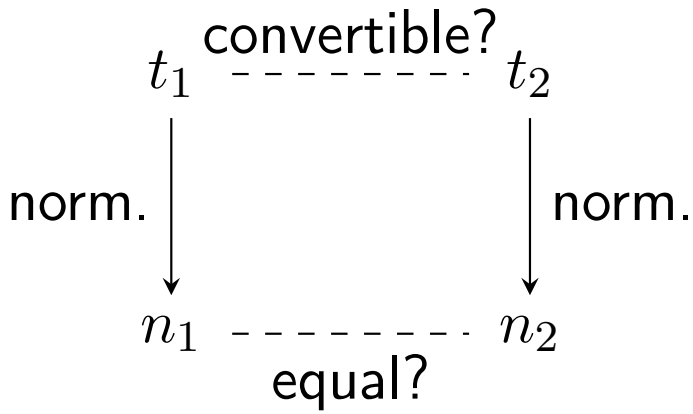
- β-reduction: $(\lambda x.t_1)\ t_2 \to_\beta t_1[x := t_2]$
- δ-reduction: if c has definition t, then $c \to_\delta t$

Convertibility test: are two terms $\beta\delta$-equivalent?

Hypothesis: only strongly-normalizing terms $\Longrightarrow$ decidability.

Testing convertibility, the easy way

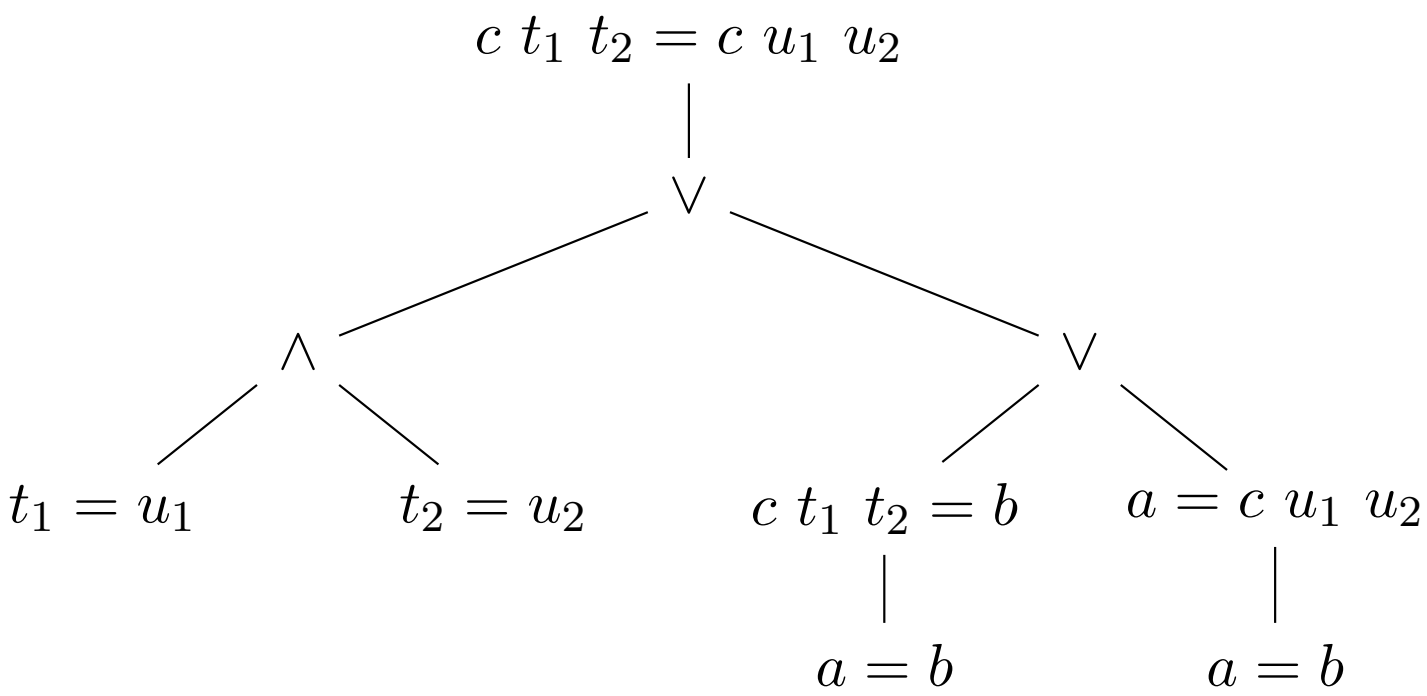
We have strong normalisation and confluence: just reduce both sides to normal form and compare the normal forms.



Computing the normal forms can be a lot of work! Sometimes we can conclude non-convertibility after computing the weak head normal forms only. For instance (without η): $x\ t \not\equiv \lambda y.u$ for all t, u.

Idea: we can mix the reduction and the comparison steps! That way, reduction of subterms happens only if necessary.

Proof tree



Exploring the proof tree

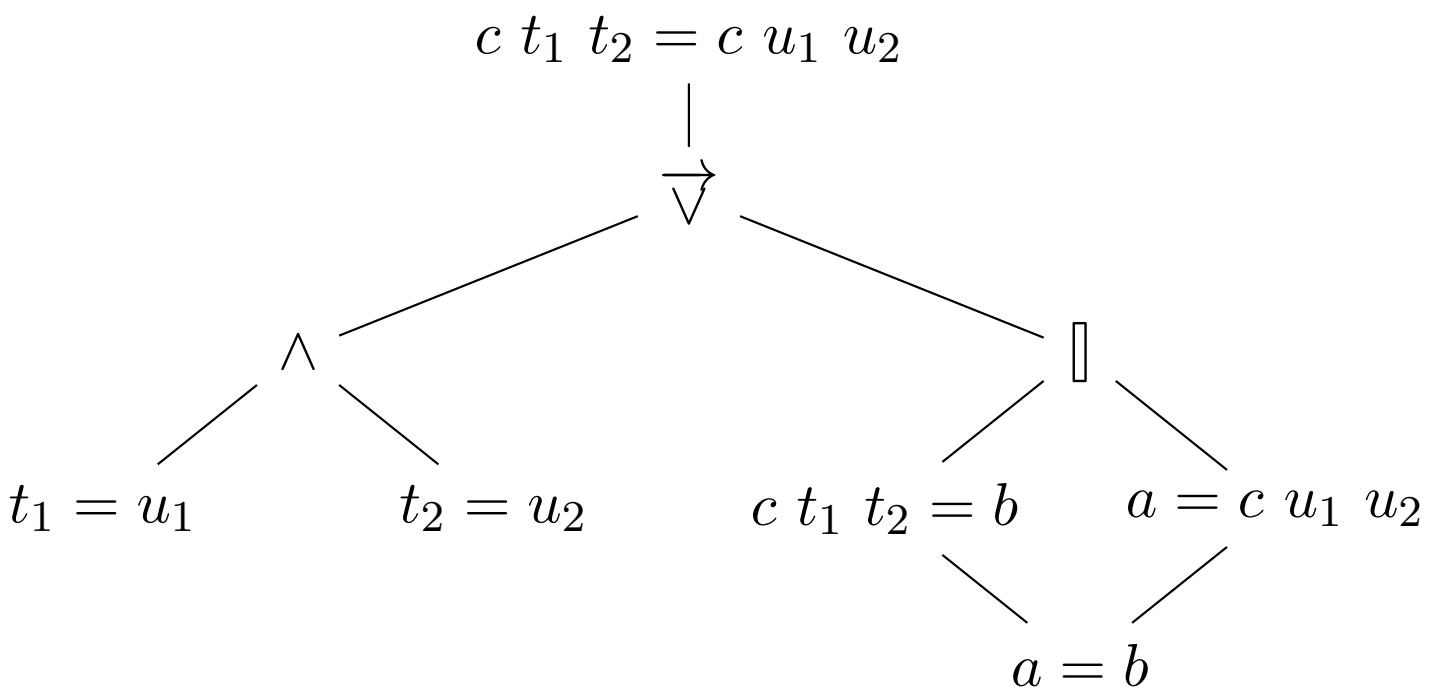
When trying to prove convertibility:

- for an "and" node, we need to prove all children,
- for an "or" node, we need to prove any child.

However: this tree is suboptimal when proving non-convertibility! Instead, we introduce two other operators $[\!]$ and $\overrightarrow{\vee}$:

- $A \,[\!]\, B$ is equal to A but assumes $A = B$, so short-circuits as soon as either A or B is computed.
- $A \overrightarrow{\vee} B$ is $A \vee B$ but assumes $A \Rightarrow B$: short-circuits if A evaluates to $\top$ or if B is computed.

Proof tree, improved



Encoding in a process calculus

Channels $\alpha, \beta, \ldots$

Processes	$P ::= \bar{\alpha}(M)$	reduce M to a value and send it on α
	$\mid C[\alpha?]$	read a value v from α, continue with $C[v]$
	$\mid P_1 \parallel P_2$	execute P_1 and P_2 concurrently
	$\mid \nu\alpha.P$	generate fresh channel name α

Communication rule:

$$C[\alpha?] \parallel \bar{\alpha}(v) \to C[v] \parallel \bar{\alpha}(v)$$

Channels and processes are used to represent non-strict evaluation:

$$C[\texttt{lazy}\ M] \approx \nu\alpha.\ (C[\alpha] \parallel \bar{\alpha}(M)) \qquad \texttt{force}\ \alpha \approx \alpha?$$

Scheduler enforces laziness: in $\bar{\alpha}(M)$, M does not reduce until α is forced.

Reduction results

Reduction is to weak head normal form, but whnfs contain lazy values for the computation of the strong normal form.

$$
\begin{aligned}
v ::= {} & \langle x, t, e, y, \alpha \rangle && \text{closures} \\
\mid {} & [x\ v_1\ \ldots\ v_n] && \text{inert terms} \\
\mid {} & [c\ v_1\ \ldots\ v_n]@\alpha && \text{defined constants}
\end{aligned}
$$

Testing convertibility, incrementally

Convertibility test $\texttt{conv}\ v_1\ v_2\ \xi$ between v_1 and v_2, using ξ as a mapping between free variables.

$$\nu\beta\gamma.\ \ \bar{\alpha}(\texttt{conv}\ \beta\ \gamma\ [\,]) \parallel \bar{\beta}(\texttt{reduce}\ t_1\ [\,]\ [\,]) \parallel \bar{\gamma}(\texttt{reduce}\ t_2\ [\,]\ [\,])$$

$\Longrightarrow$ convertibility test between t_1 and t_2, sending the result to α

Convertibility rules

For pure λ-calculus:

$$\bar{\alpha}(\texttt{conv}\ \beta\ v_2\ \xi) \to \bar{\alpha}(\texttt{conv}\ \beta?\ v_2\ \xi)$$
$$\bar{\alpha}(\texttt{conv}\ v_1\ \beta\ \xi) \to \bar{\alpha}(\texttt{conv}\ v_1\ \beta?\ \xi)$$

$$\bar{\alpha}(\texttt{conv}\ \langle x_1, t_1, e_1, y_1, \beta_1\rangle\ \langle x_2, t_2, e_2, y_2, \beta_2\rangle\ \xi) \to \bar{\alpha}(\texttt{conv}\ \beta_1\ \beta_2\ ((y_1, y_2) :: \xi))$$

$$\bar{\alpha}(\texttt{conv}\ [x\ u_1\ \ldots\ u_n]\ [y\ v_1\ \ldots\ v_n]\ \xi) \to \nu\beta_1 \ldots \beta_n.\ \bar{\alpha}\left(\xi(x) = y \wedge \beta_1? \wedge \cdots \wedge \beta_n?\right) \parallel \Vert_{i=1}^{n} \bar{\beta}_i(\texttt{conv}\ u_i\ v_i\ \xi)$$

$$\bar{\alpha}(\texttt{conv}\ v_1\ v_2\ \xi) \to \bar{\alpha}(\texttt{false})$$

all other cases

Rules for convertibility of constants

If only one head is a constant, unfold it.
If both heads are constants, we can unfold either and get the same result in any case: use the $[]$ operator to produce a result as soon as possible.

$$\bar{\alpha}(\texttt{conv}\ [c\ s]@\beta\ v_2\ \xi) \to \bar{\alpha}(\texttt{conv}\ \beta\ v_2\ \xi) \text{ if } v_2 \text{ not a constant}$$

$$\bar{\alpha}(\texttt{conv}\ v_1\ [c\ s]@\beta\ \xi) \to \bar{\alpha}(\texttt{conv}\ v_1\ \beta\ \xi) \text{ if } v_1 \text{ not a constant}$$

$$\begin{aligned}\bar{\alpha}(\texttt{conv}\ [c_1\ s_1]@\beta\ [c_2\ s_2]@\gamma\ \xi) \to \nu\delta\eta.\ & \bar{\alpha}(\delta?\ []\ \eta?) \\ & \| \bar{\delta}(\texttt{conv}\ [c_1\ s_1]@\beta\ \gamma\ \xi) \\ & \| \bar{\eta}(\texttt{conv}\ \beta\ [c_2\ s_2]@\gamma\ \xi) \\ & \text{if } c_1 \neq c_2 \text{ or } |s_1| \neq |s_2|\end{aligned}$$

Rules for convertibility of constants (cont.)

If both heads are the same constant with the same number of arguments, we can also compare the stacks, like with free variables! However terms may be convertible even if the stacks are not, so use $\overrightarrow{\vee}$.

$$\bar{\alpha}(\texttt{conv}\ [c\ u_1\ \ldots\ u_n]@\beta\ [c\ v_1\ \ldots\ v_n]@\gamma\ \xi) \to \nu\delta\eta\zeta_1 \ldots \zeta_n.$$
$$\bar{\alpha}\left(\left(\bigwedge_{i=1}^{n} \zeta_i?\right) \overrightarrow{\vee}(\delta? \parallel \eta?)\right)$$
$$\| \ \bar{\delta}(\texttt{conv}\ [c\ u_1\ \ldots\ u_n]@\beta\ \gamma\ \xi)$$
$$\| \ \bar{\eta}(\texttt{conv}\ \beta\ [c\ v_1\ \ldots\ v_n]@\gamma\ \xi)$$
$$\| \ \Vert_{i=1}^{n} \bar{\zeta_i}(\texttt{conv}\ u_i\ v_i\ \xi)$$

$\Longrightarrow$ exploration of proof tree!

More in the thesis

- Strategy for strong call-by-need
- Scheduling
- Worst-case complexity bounds
- Compilation towards a ZINC-like virtual machine
- Coq formalization

Reduction rules

$$\bar{\alpha}(\texttt{reduce}\ (\lambda x.t)\ e\ s) \to \nu\beta.\ \bar{\alpha}(\texttt{apply}\ \langle x, t, e, y, \beta\rangle\ s) \parallel \bar{\beta}(\texttt{reduce}\ t\ (e + x \mapsto [y])\ s)$$

$$\bar{\alpha}(\texttt{reduce}\ (\lambda x.t)\ e\ s) \to \bar{\alpha}(\texttt{apply}\ \langle x, t, e\rangle\ s)$$

$$\bar{\alpha}(\texttt{reduce}\ x\ e\ s) \to \bar{\alpha}(\texttt{apply}\ (e(x))\ s)$$

$$\bar{\alpha}(\texttt{reduce}\ x\ e\ s) \to \bar{\alpha}(\texttt{apply}\ [x]\ s) \quad \text{if } x \notin \mathbf{dom}(e)$$

$$\bar{\alpha}(\texttt{reduce}\ c\ e\ s) \to \nu\beta.\ \bar{\alpha}(\texttt{apply}\ [c\ [\,]]@\beta\ s) \parallel \bar{\beta}(\texttt{reduce}\ t\ [\,]\ [\,])$$

$$\bar{\alpha}(\texttt{apply}\ \beta\ s) \to \bar{\alpha}(\texttt{apply}\ \beta?\ s)$$

$$\bar{\alpha}(\texttt{apply}\ \langle x, t, e, y, \beta\rangle\ (v :: s)) \to \bar{\alpha}(\texttt{reduce}\ t\ (e + x \mapsto v)\ s)$$

$$\bar{\alpha}(\texttt{apply}\ \langle x, t, e, y, \beta\rangle\ [\,]) \to \bar{\alpha}(\langle x, t, e, y, \beta\rangle)$$

$$\bar{\alpha}(\texttt{apply}\ [x\ s_1]\ s_2) \to \bar{\alpha}([x\ (s_1 +\!\!+ s_2)])$$

$$\bar{\alpha}(\texttt{apply}\ [c\ s_1]@\beta\ s_2) \to \nu\gamma.\ \bar{\alpha}([c\ (s_1 +\!\!+ s_2)]@\gamma) \parallel \bar{\gamma}(\texttt{apply}\ \beta\ s_2)$$

Experimental evaluation

Testcase	Coq	Ours	Ours2
`test1`	3e-5	5e-5	6e-3
`test1c`	1e-5	5e-5	6e-5
`test2`	0.14	5e-6	2e-5
`test3`	9e-5	2e-4	5e-5
`test4`	0.018	0.013	9e-5
`test5`	4e-6	6e-6	8e-6
`test6`	0.61	1e-6	8e-6
`test7`	3e-5	7e-5	2e-4
`test8`	0.078	5e-5	2e-4
`test9`	2e-5	6e-5/0.18	0.15